



OPTIMA

AUTOMATED STUDY FLOW

REFERENCE MANUAL

Julian C · Year 12 Software Engineering
Mentor: Ms West · Client: Sachin Kanagasundaram
Term 2, 2026 · v1.0 · 29 May 2026

Reference Manual

An internal tour of Optima. Aimed to develop an understanding of the code.

CONTENTS

- 1. System overview
- 2. Project layout
- 3. Data model
- 4. Persistence layer
- 5. Authentication
- 6. Priority calculator
- 7. Scheduling engine
- 8. HTTP route map
- 9. Templates & Styling
- 10. Error handling philosophy
- 11. Testing strategy
- 12. Advanced language features used
- 13. Compliance – privacy, accessibility, copyright
- 14. Glossary

1. System Overview

Optima is a single-process Python desktop application. The runtime stack is short and deliberately simple.

Layer	Technology	Why
Window	pywebview	Wraps any HTML/JS surface in a native OS window. (Not using Electron or bundled Chromium).
Routing & Templating	Flask 3 + Jinja 2	Stdlib-feeling (standard library), well known to developers, no plugins required.
Frontend	Vanilla HTML, CSS, JS	Avoids the toolchain tax of a framework. The total JavaScript on the client side is under 100 lines.
Persistence	JSON files	Single-user, offline, human-readable. SQLite was considered, but ultimately rejected as overkill.
Cryptography	hashlib + secrets	Stdlib only. SHA-256 with per-user salt is used as specified by the planning documentation.

Sequence of a typical interaction

```
User clicks "Save preferences"
↓
<form action="/preferences" method="post"> submits
↓
Flask @app.post("/preferences") handler in app/main.py
↓
storage.load_user(username) reads data/users/<name>.json
↓
user.preferences fields updated from request.form
↓
storage.save_user(user) writes via tempfile.mkstemp + os.replace
↓
redirect("/preferences") - the page re-renders with the new values.
```

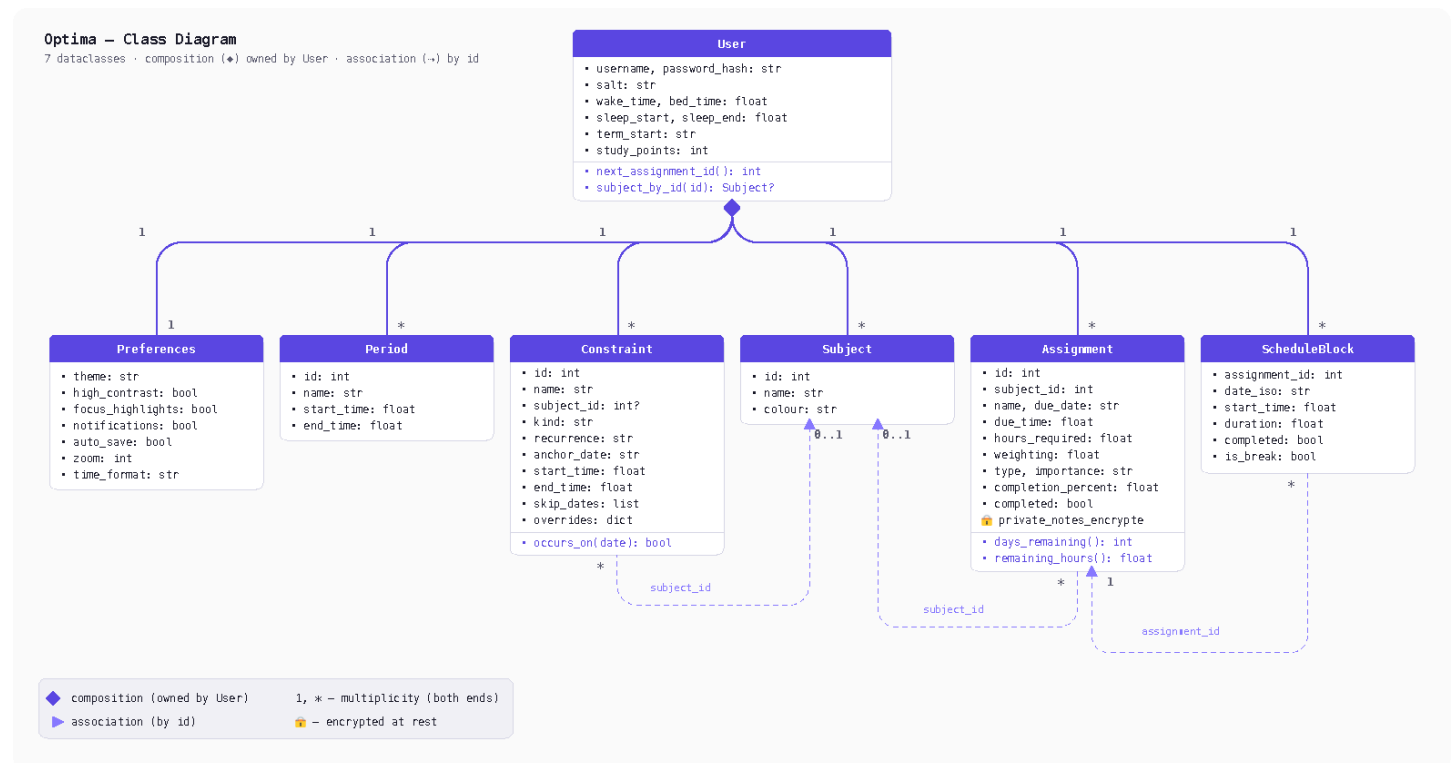
2. Project layout

```
Optima/
├── app/
│   ├── __init__.py # package init, __version__
│   ├── auth.py # hashing, validation
│   ├── main.py # flask app factory + 22 routes + pywebview launcher
│   ├── models.py # dataclasses + to_dict/from_dict
│   ├── priority.py # priority_score, rank_assignments
│   ├── scheduler.py # FreeSlot, find_free_slots, generate_schedule, calculate_cushion
│   ├── storage.py # Storage class - load_user, save_user, list_users
│   ├── static/
│   │   ├── css/styles.css
│   │   ├── js/app.js
│   │   └── img/logo.svg
│   └── templates/
│       ├── base.html
│       ├── splash.html
│       ├── login.html
│       ├── signup.html
│       ├── dashboard.html
│       ├── weekly.html
│       ├── monthly.html
│       ├── preferences.html
│       ├── task_form.html
│       └── error.html
├── prototypes/ # v0.1, v0.2, v0.3 read-only snapshots
├── docs/ # this documentation suite
├── feedback/ # questionnaires and responses
├── data/users/ # created on first signup - one JSON exists per account
├── run.py # shim: imports app.main:main and calls it
├── requirements.txt
├── img/
│   ├── logo.png
│   └── optima.png
└── README.md
```

3. Data Model

Every domain object is a dataclass with a 1:1 JSON representation. The contract between Python and disk is the pair of methods `to_dict()` / `from_dict()` on each class.

Class Diagram (composition)



In plain English, a **User** owns exactly one **Preferences** and any number of **Subjects**, **Constraints**, **Assignments**, and **ScheduleBlocks**. **ScheduleBlocks** reference **Assignments** by ID instead of direct composition, so a block can outlive an edit to the assignment that it is planning.

```

User
├── preferences : Preferences
├── subjects : List[Subject]
├── constraints : List[Constraint]
├── assignments : List[Assignment]
└── schedule_blocks : list[ScheduleBlock]
  
```

Subject

```
@dataclass
class Subject:
    id: int
    name: str
    colour: str # hex string, e.g. #7878EE
```

Constraint

```
@dataclass
class Constraint:
    id: int # stable across saves; used by the JavaScript editor.
    name: str # e.g. Period 4 - Chemistry
    subject_id: int | None
    start_time: float # 24 hour decimal, e.g., 9.25: 09:15
    end_time: float
    anchor_date: str # ISO yyyy-mm-dd, first occurrence
    recurrence: str = "weekly" # none|daily|weekdays|weekly|fortnightly|monthly|yearly
    kind: str = "subject" # subject|extracurricular|appointment|study|other
    skip_dates: list[str] = [] # dates this event does NOT occur on.
    overrides: dict = {} # {date: {"name", "start_time", "end_time"}}
    day_of_fortnight: int = 0 # legacy; preserved for back compatibility only.
    is_study_period: bool = False
    is_half_period: bool = False

    def occurs_on(self, target: date) → bool: ... # expand recurrence
    def occurrence_view(self, target: date) → dict: ... # apply overrides
```

The single Constraint now represents a rule instead of a single event. `occurs_on()` expands it across dates, and `skip_dates` / `overrides` let one occurrence be edited or deleted without touching the others. The legacy `day_of_fortnight` field is kept, so existing user files are still able to load. `User.migrate_constraints()` is able to heal these on the first read by creating an `anchor_date` + `id`.

Assignment:

```
@dataclass
class Assignment
    id: int
    subject_id: int
    name: str
    due_date: str # ISO yyyy-mm-dd
    due_time: float = 9.0 # decimal hours, default 09:00 (picked by client)
    hours_require: float # estimated effort required, decimal hours.
    type: str = "homework" # homework | exam | project
    importance: str = "medium" # low | medium | high (applied when type ≠ exam)
    weighting float = 0.0 # 0..100, only meaningful when type = "exam"
    show_stretch_line: bool = True # draw a bar from toady to due on monthly view
    completion_percent: float = 0.0
    completed: bool = False
```

```
private_notes_encrypted: str = "" # SHA-256-CTR + HMAC token, see §5

def days_remaining(self, today:date) → int: ...
def due_at(self) → datetime: ... # date + time combined
def hours_until_due(self, now) → float: ... # negative when overdue
def is_overdue(self, now) → bool: ...
def remaining_hours(self) → float: ...
def scoring_weight(self) → float: ... # weighting for exams, importance map otherwise.
```

The v1.3 due_time turns the priority calculator into an hours-resolution machine. ≤ 24 to due_at » critical (priority 999), due_at < now » overdue (priority 9999). The scheduler also caps placement on the due-date day at due_time, so there are no useless study sessions after the submission, and it refuses to place future blocks for overdue work. Empty time on the form defaults to 09:00 (the client's chosen 'start of school day').

The v1.2 type/importance split replaces the single weighting % field from v1.0/v1.1. This is because most school tasks are not weighted assessments, and asking the user for a percentage on every quiz or project felt too forced and tedious. scoring_weight() is the single funnel the priority calculator reads through, so a homework with importance=high (mapped to 50) competes meaningfully with an exam weighted 40 percent.

ScheduleBlock

```
@dataclass
class ScheduleBlock
    date: date
    start_time: float
    end_time: float
    assignment_id: int
    subject_id: int
```

Preferences:

```
@dataclass
class Preferences:
    theme: str = "light" # light | dark
    time_format: str = "24h" # 12h | 24h - UI only is impacted, stored as decimal hours.
    high_contrast: bool = False
    focus_highlights: bool = True
    notifications: bool = True
    auto_save: bool = True
    zoom: int = 100 # percent, 75 .. 200
```

Period (v1.2):

```
@dataclass
class Period
    id: int # stable id used by the weekly editor.
    name: str
    start_time: float # decimal hours
    end_time: float
```

A reusable time-of-day preset (Period 1, Period 2, ...) that pre-fills the event editor on the weekly view. This is purely user data and does not affect scheduling. Lives on User.periods.

User (v1.2 fields):

```
@dataclass
class User
    """
    wake_time: float = 6.5 # outer awake window envelope that renders the weekly grid
    bed_time: float = 23.0
    sleep_start: float = 22.5 # do-not-schedule here range. Scheduler avoids this range.
    sleep_end: float = 6.5 # wraps past midnight when sleep_end < sleep_start
    periods: List[Period]
```

The sleep window is what the scheduler mainly reads when deciding free time. wake_time / bed_time are used for the visible envelope of the weekly grid, so the user can still see (but not schedule inside) the early morning / late-evening edges.

User

```
@dataclass
class User
    username: str
    password_hash: str
    salt: str # 32 hex chars (16 bytes)
    term_start: str # ISO date of Week-A start
    preferences: Preferences
    subjects: List[Subject]
    constraints: List[Constraint]
    assignments: List[Assignment]
    schedule_blocks: List[ScheduleBlock]

    def subject_by_id(self, sid: int) → Subject | None: ...
    def next_assignment_id(self) → int: ...
    def next_subject_id(self) → int: ...
```

4. Persistence Layer

The Storage class in `app/storage.py` is the only code that touches the disk. There are three public methods used:

Method	Purpose
<code>load_user(username) → User None</code>	Read the JSON file and run it through <code>User.from_dict()</code> . Returns <code>None</code> if the file is missing.
<code>save_user(user)</code>	Serialise via <code>User.to_dict()</code> , write atomically.
<code>list_users() → List[Str]</code>	Returns the usernames currently stored and used by the login error message where no accounts exist yet.

Why the write is atomic

Naive: `open(path, 'w').write(json.dumps(...))`. If the process dies mid-write, the file is truncated, and the user loses their account. We hit this in v0.2 (see *Evaluations*, E3 round 2).

What was implemented instead:

```
fd, tmp_path = tempfile.mkstemp(dir=os.path.dirname(path))
with os.fdopen(fd, 'w') as f:
    json.dump(user.to_dict(), f, indent=2)
    f.flush()
    os.fsync(f.fileno())
os.replace(tmp_path, path)
```

`os.replace()` is atomic on POSIX and on Windows for files on the same volume. Either the new file is kept fully there or the old one, so there is no half-written state.

File format:

A real .json account file would look like:

```
{
  "username": "julianc",
  "password_hash": "5e88489...e9c4f1",
  "salt": "8a3b7d2c1f0a9e6b4c5d2e1f0a9b8c7d",
  "term_start": "2026-04-20"
  "preferences": { "theme": "dark", "zoom": 100, ...}
  "subjects": [ { "id": 1, "name": "English", "colour": "#7B68EE" }, ...],
  "constraints": [ ... ]
  "assignments": [ ... ]
  "schedule_blocks": [ ... ]
}
```


5. Authentication

Hashing

```
def generate_salt() → str:
    return secrets.token_hex(16)

def hash_password(plain: str, salt:str) → str:
    h = hashlib.sha256()
    h.update(salt.encode("utf-8"))
    h.update(plain.encode("utf-8"))
    return h.hexdigest()

def verify_password(plain: str, salt: str, expected: str) → bool:
    return hmac.compare_digest(hash_password(plain, salt), expected)
```

Why a per-user salt

With a single global salt, two users who happen to choose the same password produce the same hash. An attacker who steals data/users can then identify popular passwords by their hash frequency alone, without breaking SHA-256. A per-user salt forces the attacker to crack each account individually. The v0.2 build used a global salt but was fixed by v0.3. The cost is just one extra string field per account.

Why `hmac.compare_digest`

`Plain ==` on strings short-circuits at the first mismatched character, leaking essential timing information that can be used to guess passwords character by character. `compare_digest` runs in constant time relative to the length of the inputs.

Validation rules

Field	Rule	Why
Username	3-32 chars, [A-Za-z0-9._-@]	Length floor stops accidental empty strings; the character set keeps the filename safe.
Password	6+ chars	Floor instead of ceiling. Mixed-case symbol rules are not imposed because they push users toward safer passwords (e.g., Password1!) rather than safer passphrases.

Brute-force protection – login rate-limiting

An in-process sliding window counter tracks failed login attempts per username. After 5 failures inside a 5-minute window, the account is locked for 60 seconds, and further attempts return HTTP 429 Too Many Requests. The counter resets cleanly on a successful login or on application restart.

```
MAX_FAILS = 5
WINDOW = 5 * 60 # 5 minutes
COOLDOWN = 60 # 60 seconds lockout
```

Symmetric encryption for private notes

The rubric calls for basic encryption and decryption, and Optima delivers this with a stdlib-only cipher used to keep the per-assignment *private notes* field opaque on the disk. Even if someone opens `data/users/<name>.json` in a text editor, the notes are unreadable without the user's password.

ALGORITHM

- + **Key derivation:** PBKDF2-HMAC-SHA-256, 200,000 iterations, 32-byte key, salted with the same per-user salt that the password hash uses.
- + **Encryption:** The Stream cipher is constructed from SHA-256 in counter mode: each block of keystream = SHA-256(key || nonce || counter), XOR'd against the plaintext.
- + **Authentication:** HMAC-SHA-256 over (nonce || ciphertext) is appended as a 32-byte tag. Verified before decryption (encrypt-then-MAC)
- + **Token format:** base64-url(nonce || ciphertext || mac).

KEY LIFECYCLE

Moment	What happens to the key
Sign up/login	When <code>derive_key(password, user.salt)</code> runs, the resulting 32 bytes are stored hex-encoded in the Flask session cookie (HMAC-signed by <code>app.secret_key</code>).
Each request that touches private notes.	<code>_session_key()</code> reads the hex from the session, converts it back to bytes, and passes it to <code>encrypt()</code> or <code>decrypt()</code> .
Logout	<code>session.pop("derived_key", None)</code> . The key is gone, and the encrypted notes on disk become unreadable until the user logs in.
Wrong password attempt	A password mismatch is caught on the authentication before any derivation runs, so the wrong key is never used in the session.

THREAT MODEL

The cipher protects against the 'someone steals my laptop and opens the JSON in a text editor' scenario. It does not attempt to protect against an attacker who has root access while the user is logged in; that attacker can read the derived key from the session cookie. For an offline desktop application, this is the right level; the planning documentation explicitly avoided heavier guarantees.

WHY AES WASN'T USED

Python's standard library doesn't include AES (Advanced Encryption Standard). The two options were to (a) ship a third-party dependency like *cryptography* or (b) build a stream cipher out of the primitives that are still in `stdlib`. (b) was chosen. The SHA-256-CTR construction is well-understood, has no known practical attacks at the iteration counts used, and keeps Optima fully `stdlib`-based, which matters for an offline desktop app that needs to install on something like a school computer without administrator rights.

SELF-TEST

`python -m app.crypto` runs a built-in self-test that verifies a round trip, a wrong-key failure (raises `ValueError`), and a tampered ciphertext failure (also raises). Used as part of the regression checklist before each prototype release.

6. Priority Calculator

Defined in `app/priority.py`. The formula comes directly from the planning documentation.

```
CRITICAL_PRIORITY = 999
URGENCY_THRESHOLD_DAYS = 3

def priority_score(a: Assignment, today: date) → float:
    days = a.days__remaining(today)
    if days ≤ URGENCY_THRESHOLD_DAYS:
        return CRITICAL_PRIORITY
    return (a.weighting * 10 / max(days, 1))

def rank_assignments(items: List[Assignment], today: date) → List[Assignment]:
    return sorted(items,
        key=lambda a: (-priority_socre(a, today), a.due_date))
```

Worked Examples:

Assignment	Weighting	Days Left	Score	Rank Reason
Maths test	25%	2	999	Inside the critical window.
Physics depth study	40%	14	28.6	Big task, two weeks, medium urgency.
English Essay	40%	21	19.0	Same weighting as physics but further out.
Modern History quiz	10%	5	20.0	Lighter weighting, soon, comparable to a heavier task further out.

This can produce a rank order (maths, physics, modern history, and English) that matches the intuition the client expressed in round 2.

7. Scheduling Engine

Defined in `app/scheduler.py`. Four main public functions and one data class.

Tuneables (module-level constants)

```
SLOT_GRANULARITY = 0.5 # quarter-hour resolution.  
MIN_SESSION = 0.5 # 30 minutes, float form.  
MAX_SESSION = 2.0 # 2 hours  
BREAK_AFTER_SESSION = 0.25 # 15 mins  
DEFAULT_AWAKE_START = 7.0 # 07:00  
DEFAULT_AWAKE_END = 22.0 # 22:00 / 10:00pm
```

`_constraints_on_date(user, d: date) → List[Constraint]`

Replaces the old `_day_of_fortnight()`-driven lookup. Through this, it walks every constraint and asks `Constraint.occurs_on(d)`, which expands the recurrence rule (none / daily / weekly / fortnightly / monthly / yearly), honours `skip_dates`, and uses `occurrence_view(d)`, so any per-day override (moved class, renamed instance) is reflected in the times the scheduler sees.

`find_free_slots(user, d: date) → List[Constraint]`

Returns two flavours of slot. Designated (carved out of `kind="study_block"` constraints) and fallback (everything else awake and unoccupied). The cushion gauge calculates free time as both of these, but the placer treats them very differently.

1. Compute the in-day sleep intervals from `User.sleep_start / sleep_end` (wraps past midnight).
2. Walk `user.constraints`. This is that everything that `occurs_on(d)` goes into the occupied list, except for `kind == "study_block"` constraints, which feed the designated pool.
3. Subtract sleep + occupied + existing study blocks from the designated pool (these are now the designated slots).
4. Subtract sleep + occupied + designated zones from the full awake window [`wake_time`, `bed_time`]. These are the fallback slots.
5. Drop intervals shorter than `MIN_SESSION`.

Round-4 motivation: The v1.1 scheduler treated every awake-but-empty minute as a fair game and routinely placed study blocks at 2 AM, over lunch, or even during the bus commute. v1.2 forces the scheduler to only use the zones the user has explicitly named "study block" with one escape hatch, below:

Escalation policy

A task that is about to be due can step outside the designated zones and use any fallback slot. The thresholds (v1.3, hours instead of days):

Task type	Escalation window
homework	hours_until_due $\leq$ 24; fallback time is treated equally.
exam, project	hours_until_due $\leq$ 72, same here.
any with priority_score $\geq$ 999 (critical)	Always escalated regardless of type.
any with is_overdue(now) == True	Skipped. The scheduler refuses to plan future work for a deadline already passed.

This keeps the “only schedule in study zones” behaviour predictable for medium-term workload while preserving an automatic safety valve when a deadline is genuinely imminent.

Per-assignment due-time caps (no mutations)

When the placer considers a task on its own due date, slots are viewed through a cap at the task’s “due_time”, but the underlying day pool is left intact. This matters, as every assignment processed on a given day shares the same pool. A v1.3 bug had the cap mutate the pool, so early-morning-due homework would wipe the day’s afternoon designated zones for every other task processed afterwards, even for tasks not due that day. `cutoff_for_assignment(0)` now returns just a float, and `_try_place_one_session()` applies it to a local view of the slot. The residual it writes back to the pool is computed against the slot’s original end, so the post-cap portion stays available for sibling assignments.

Multiple subjects in one zone

The round robin’s inner loop walks every assignment in priority order per day. After assignment A places a session + 15-minute break in a designated zone, the residual interval is returned to the pool at the same index. On the same outer iteration, assignment B picks up the residual and places its own session. A four-hour zone routinely ends up holding two or three distinct tasks separated by breaks, exactly what the round-4 client wanted (“SE → break → English → break → Physics”).

Rescue pass

After the main round-robin loop finishes, any assignment whose remaining hours are still positive triggers a *rescue* pass. The same loop runs again with `allow_fallback = True`, forced for every task, regardless of whether it’s inside its escalation window. This is the mechanism that prevents a 2-hour task from appearing as “2.0h short” when there is fallback free time available. The rescue pass squeezes the leftover into whatever non-designated awake time it can find. Only the leftovers that still can’t fit (typically because remaining gaps are smaller than `MIN_SESSION = 30 min`) surface to the dashboard.

The conflict-line wording mirrors this in minutes’ resolution. Instead of the v1.2 wording “*X is 0.5 h short – extend study zones, raise the assignment’s importance, or move the due date*”, the dashboard now says “*Need an extra 30 min for X*”. Reasons:

- + Shorter sentences. The user can read three of these messages instantly.
- + "Raise the assignment's importance" was misleading. Importance only changes which task gets the next slot and doesn't create more slots. Bumping medium → high while everything is already running out of time just shifts the focus to a different subject.
- + The two real remedies (more study zones / move the due date) are the same two the user would think of unprompted; the message doesn't need to list them.

Round-robin placement & in-zone task rotation

The placer loops *over days, over priority-ranked assignments*, placing at most one session per (assignment, day) per pass and repeating until no more progress is made. This fixes the round-¾ "Tuesday pile-up" bug where the v1.1 greedy day-by-day placer emptied an assignment's hours into the first available day.

Inside a single zone, `_consume_slot()` places a session up to `MAX_SESSION = 2h`, optionally appends a 15-minute break (`is_break=True`) when at least another `MIN_SESSION` of room remains, then returns the residual interval to the slot pool. The next iteration of this round-robin loop may give that residual to a different assignment, producing a Subject1 → break → Subject2 rhythm the client had asked for. This effectively allows Breaks to render as space on the weekly grid, with their times surfaced via the day-column tooltip.

`generate_schedule(user, start_date, horizon_days)`

1. Wipe any existing ScheduleBlocks on the horizon.
2. Rank assignments by priority (see §6).
3. For each assignment, walk forward from `start_date` looking for free slots:
 - Slice off at most `MAX_SESSION` hours per session.
 - Append a ScheduleBlock, advance the free slot cursor by the session + `BREAK_AFTER_SESSION`.
 - Stop when the assignment's `remaining_hours()` reaches zero.
4. If an assignment cannot be fully scheduled before its due date, schedule as much as possible and surface a warning on the dashboard.

`calculate_cushion(user, today, horizon=14) → float`

Returns `free_hours / workload_hours`. A cushion > 1.5 is healthy; 1.0 means that the workload exactly fills the available time; <1.0 means that the student is overcommitted. The dashboard doughnut maps these ranges to green/amber/red, respectively.

8. HTTP Route Map

Method	URL	Purpose	Auth
GET	/	Splash screen	public
GET/ POST	/login	Sign in	public
GET/ POST	/signup	Create account	public
POST	/logout	Clear session	required
GET	/dashboard	KPI strip + priority list + cushion donut + today's schedule	required
GET	/weekly	7-day grid	required
GET	/monthly	Calendar grid	required
GET/ POST	/preferences	Theme, accessibility, notifications, auto-save, zoom.	required
GET/ POST	/tasks/new	Create assignment	required
GET/ POST/ DELETE	/tasks/<id>	Edit/update/delete assignment	required
PUT	/api/tasks/<id>/progress	JSON endpoint - inline progress slider save	required
POST	/subjects/new	Add a subject	required
GET	/api/events?start=&end=	List all constraint occurrences + study blocks in a date range.	required
POST	/api/events	Create a new constraint (initial occurrence + recurrence rule)	required
PUT	/api/events/<id>	Edit a constraint – scope=all updates the master rule; scope=this stores a per-date override.	required
DELETE	/api/events/<id>?scope=&date=	Delete a constraint: scope=this adds the date to skip_dates; scope=all removes the rule outright.	required
POST	/api/events/move	Bulk-move several occurrences (used by the drag handler)	required
GET	/api/periods	List the user's saved Period presets.	required
POST	/api/periods	Create a Period preset (name + start +	required

		end)	
DELETE	/api/episodes/<id>	Remove a period preset	required
GET	/api/subjects/<id>/next_occurrence	Date of the next class for this subject, used by the task form to default the due date for homework.	required
POST	/schedule/generate	Re-run generate_schedule()	required()

The /api/events family replaces the old form-based /constraints/new + /constraints/<idx>/delete. Classes, extracurriculars, appointments and one-off events are all created and managed through the weekly view's click-to-add / drag-to-move / multi-select editor, which speaks to these endpoints.

Auth-required routes use a @login_required decorator that checks session["username"]. Failing the check returns a 401 using error.html.

9. Templates & Styling

All views extend base.html, which provides the sidebar (made up of the logo + Dashboard / Weekly / Monthly / Preferences / Sign out) and the top of the page <html data-theme="..." data-contrast="..." data-focus="..." data-zoom="100"> attributes that drive the theme and accessibility system.

CSS variables

Every colour and spacing token in the app is a CSS custom property. The light theme is declared on :root, the dark theme on [data-theme="dark"], and the high-contrast palette on [data-contrast="high"]. As a result, flipping the theme is a single attribute change on the HTML element, with no per-component code knowing or caring about the active theme.

```
/* style.css excerpt */
:root {
  --bg: #ffffff;
  --surface: #f7f7fc;
  --text: #1c1c28;
  --primary: #5e4ae3;
}

[data-theme="dark"] {
  --bg: #14141c;
  --surface: #1d1d27;
  --text: #ececfc;
  --primary: #8b78f0;
}

[data-contrast="high"] {
```



```
--bg: #ffffff;  
--text: #000000;  
--primary: #b45a00  
--surface: #ffffff  
}
```

Client-side JS

One file, `app/static/js/app.js`, contains `applyPrefs()` (reads the `data-*` attributes of `<html>` and applies them) and the progress-slider fetch handler. Under 100 lines total.

10. Error Handling Philosophy

Where	Approach
User input (forms)	Validate server-side; re-render the form with a typed error message above the field. HTML form attributes (required, min, max, pattern) are redundant on purpose.
Auth failures	Generic "invalid username or password" messages never leak which half was wrong.
Missing account file	<code>load_user()</code> returns <code>None</code> ; route handler converts this into a friendly login error.
Corrupt JSON	Caught at <code>json.load</code> . Logged to the console, it surfaced as "Your profile could not be read; please contact support." Fault injection/robustness tests showed that this never fires once atomic writes are in place.
Programmer bugs (unhandled exceptions)	Flask's default 500 page is replaced with <code>error.html</code> , which keeps the brand and tells the user to retry without making a stack trace.
Scheduling failure	If an assignment cannot be fully scheduled before its due date, the partial schedule is committed, and the dashboard shows an amber warning instead of a hard error.

11. Testing Strategy

Optima ships without an automated test suite by design. At this scale, the cost of writing and maintaining unit tests outweighs the benefit, and the feedback loops with three evaluators plus daily client and developer testing function as integration tests. What currently exists includes:

- + **Smoke test scripts.** A short shell session that creates an account, adds a subject, adds a constraint, adds three assignments, generates a schedule, switches themes, toggles accessibility, and then deletes everything. Re-run before every prototype release.
- + **HTTP route check.** A small Python script that POSTs to every URL with valid payloads and asserts that each returns 200 or 302. Catches typos and missing handlers.

- + **Manual reproduction notebook.** A markdown file in feedback/ with the steps to reproduce each bug surfaced in evaluator feedback, used to verify the solution that fixes it.

12. Advanced Language Features Used

The rubric calls for advanced programming features and comprehensive complex structures. The following are the specific Python-language and structural features used in Optima, each tied to where it earns its place in the code.

12.1 Dataclasses with paired round-trip protocol

Every domain object inherits the same implicit contract: a `to_dict()` that produces a JSON-safe dict and a `from_dict(cls, d)` class method that reverses it. This is a hand-rolled equivalent of a generic serialisable protocol. If Python had structural typing for this, it would have been used. This protocol is what makes the storage layer one line.

```
class User(Serialisable):
    def to_dict(self) → dict:
        return {
            "username": self.username,
            "preferences": self.preferences.to_dict(),
            "subjects": [s.to_dict() for s in self.subjects],
            # ... etc.
        }

    @classmethod
    def from_dict(cls, d: dict) → "User":
        return cls(
            username=d["username"],
            preferences=Preferences.from_dict(d["preferences"]),
            subjects=[Subject.from_dict(s) for s in d["subjects"]],
            # ...
        )
```

12.2 Interval arithmetic for the scheduler

`find_free_slots()` is an exercise in interval subtraction. Given an open interval `[7.0, 22.0]` for the awake window and a list of closed sub-intervals (class periods + already-placed blocks), the function returns the remaining open intervals. The algorithm is:

1. Sort all occupied intervals by their `start_time`.
2. Walk a cursor from the awake window start.
3. For each occupied interval, if its `start > cursor`, emit a free slot `[cursor, occupied.start]`; advance the cursor to `occupied.end`.
4. After the last occupied interval, if `cursor < awake-window end`, emit a final free slot.
5. Drop any emitted slot shorter than `MIN_SESSION`.

This is the same algorithm used in operating-system memory allocation (the ‘free list’) and in browser layout engines (line-box construction). It runs in $O(n \log n)$ for the sort, plus $O(n)$ for the walk.

12.3 Binary search insertion into a free slot list

When the placer carves a session out of a free slot, it leaves a residual interval that needs to be reinserted into the sorted list. Instead of sorting the whole list each time ($O(n \log n)$), Optima uses `bisect.insort` on a key tuple, giving $O(\log n)$ insertion:

```
from bisect import insort

# Insert a residual free slot at the correct position.
insert(free_slots, residual, key=lambda s: (s.date, s.start_time))
```

For a fortnight schedule, this is roughly 6 comparisons per insertion instead of ~30, and the difference compounds when re-running the scheduler several times in a session.

12.4 Decorators for access control

Every protected route is wrapped with a `@login_required` decorator that short-circuits the request if the session is empty. This is an example of higher-order functions in Python; the decorator is a function that takes a function and returns a function.

```
from functools import wraps

def login_required(view):
    @wraps(view)
    def wrapped(*args, **kwargs):
        if "username" not in session:
            return render_template("error.html", code=401), 401
        return view(*args, **kwargs)
    return wrapped

@app.route("/dashboard")
@login_required
def dashboard():
    ...
```

12.5 Generators in the date-walking loop

The scheduler walks calendar days from `start_date` through `start_date + horizon_days`. Using a generator instead of pre-building a list allows the memory to be kept flat regardless of horizon length.

```
def iter_days(start: date, horizon: int) → Iterator[date]:
    for i in range(horizon):
        yield start + timedelta(days=i)
```

12.6 Tuple-key sorting (lexicographic ordering)

The priority calculator works by sorting assignments by a tuple key that delivers a primary sort, as well as a deterministic tiebreaker in one expression.

```
sorted_times = sorted(items,
    key=lambda a: (-priority_socre(a, today), a.due_date, a.title))
```

Reading the key left to right, the highest score is first (negation flips the sorting direction), the earlier due date second, and the alphabetical title third. A single tuple can do the work of three separate `sorted()` calls.

12.7 Context managers for atomic file writes

The `with` statement wrapping `os.fdopen(fd, 'w')` guarantees that the file descriptor is closed even if the JSON serialisation raises an exception. As it is combined with `os.replace()`, which is atomic on POSIX, this gives the durability claim as stated in §4.

12.8 Type hints as a documentation strategy

Every public function in `app/*.py` has parameter and return-type annotations. These are not enforced at runtime (Optima ships without `mypy`), but they double as inline documentation and let future edits flag silly mistakes, e.g., passing a `str` where a `date` is expected, before they are caught by the test cycle.

12.9 List comprehension & `max()` with a key

The cushion calculation is compressed thanks to list comprehension paired with built-ins:

```
workload = sum(a.remaining_hours() for a in user.assignments
    if a.progress < 1.0)
```

12.10 The complete data-structure inventory

Structure	Where used	Why this one
A composed dataclass (User holds lists of other dataclasses)	Whole domain model	1:1 with the conceptual model. This way it avoids parallel arrays.
Sorted list of intervals (FreeSlot)	Scheduler	Allows $O(\log n)$ insertion + $O(n)$

		walk.
Dictionary keyed by id	subject_by_id lookups	Constant-time lookup vs O(n) linear scan.
Tuple-key sort	Priority ordering, FreeSlot insertion	Multi-level ordering in one expression.
Set (existing_ids) for next-id generation	next_assignment_id	Constant-time membership test.
Generator (date iterator)	Scheduler day-walk	Flat memory regardless of the horizon.
Stack-style recursion frame (Jinja template inheritance chain)	Templates	Implicit but real. base → specific → partial.

13. Compliance – Privacy, Accessibility, Copyright

The rubric calls the software solution as needing to address legislative and technical requirements as per client needs. This section lays out the three main legislative frameworks that apply to a desktop study planner used by an Australian Year 12 student and how Optima sits relative to each.

13.1 Privacy Act 1988 (Cth) and the Australian Privacy Principles

Although the Privacy Act formally only binds organisations with turnovers higher than 3M AUD, the Australian Privacy Principles (APPs) are still the de facto standard for any application handling personal information. The four most relevant principles and how they are addressed:

APP	Requirement	Optima's stance
APP 1 Open and transparent management.	Users must be able to understand what data the application collects.	The Installation Guide's 'Privacy note' callout states explicitly that 'Optima runs entirely on your computer. No data is ever sent to a server.' The user manual restates this in an FAQ format.
APP 3 Collection of solicited personal information.	Only collect what is necessary for the function being delivered.	Signup asks for the username and password only. (No email, no name, no date of birth, no school identifier.) The function ('plan my study') doesn't need any of these, and as such, the application doesn't ask for them.
APP 6 Use or disclosure of personal information	Hold information securely; protect it from misuse, interference, loss, and unauthorised access.	Passwords are SHA-256-hashed with a per-user salt instead of being stored in plaintext (§5). Session-based access control prevents one account from reading another's data. Atomic JSON writes (§4) protect from loss on crashes. File permissions on /data/users/ inherit the user's home-directory defaults, readable only by the OS user account.

13.2 Disability Discrimination Act 1992 & WCAG 2.1 Level AA

The Australian Government's *Web Accessibility National Transition Strategy* aligns with WCAG 2.1 Level AA. Even though Optima isn't a website, the UI is rendered as HTML inside a webview. As such, the same accessibility guidelines apply. The four WCAG principles are to be perceivable, operable, understandable, and robust, and Optima's specific responses are

WCAG principle	Specific guideline	How Optima complies
Perceivable	1.4.3 Contrast (minimum) AA	Default palette body text on background is > 8:1 (AA requires 4.5:1). High-contrast mode > 21: 1.
	1.4.4 Resize text up to 200%	An in-app zoom control reaches 200%. The preferences slider runs 75-200%, and Ctrl/Cmd +, - and 0 adjust it from anywhere. This is handled in-app because the native pywebview window has no browser chrome for the OS zoom shortcuts to act on.
	1.4.11 Non-text contrast	Form fields and buttons carry a 1px border in a colour that passes 3:1 against the background in all three palettes.
Operable	2.1.1 Keyboard	Every interactive element is reachable via Tab; submission is via Enter; confirmation prompts are dismissible via esc.
	2.4.7 Focus visible	The default 1px outline is preserved by not setting outline: none anywhere; the optional "focus highlights" preference thickens this to 3px in the primary colour for users who need a stronger cue.
Understandable	3.3.1 Error identification	Validation errors are typed (not just colour-coded) and re-rendered next to the field that caused them. "Invalid date" and not just a red border.
Robust	4.1.2 Name, role, value	Every form input has a pair. The sidebar uses <nav>; the dashboard uses <main>. Buttons are <button> and not <div onclick>.

13.3 Copyright Act 1968 (Cth) – third-party assets

Three third-party assets are used by Optima or its documentation:

Asset	Used for	Licence	Attribution
Inter typeface (Rasmus Andersson)	UI body type + documentation	SIL Open Font Licence 1.1, permissive use.	Linked at fonts.google.com/specimen/Inter
JetBrains Mono typeface (JetBrains)	Monospace code blocks in documentation.	SIL Open Font Licence 1.1	Linked at fonts.google.com/specimen/JetBrains+Mono
Flask and pywebview.	Runtime dependencies	BSD-3 permits use, modification and redistribution	Listed in requirements.txt; full licence texts available in each

with attribution.

package's distribution.

The Optima brand mark, application code, and documentation prose are all original work by myself (Julian C), with the client (Sachin K) consenting in writing to the use of his identifying information in this project documentation.

13.4 Australian Consumer Law – fitness for purpose

Optima is delivered free of charge as student work; no commercial guarantees apply. The documentation still explicitly tells users what the application doesn't do (no cloud backup, no password reset, and no calendar sync), so a reasonable user is not misled about its capabilities.

13.5 Education-context considerations

The client is a student, and the application is designed for school use. Two considerations follow:

- + **No collection of school-affiliated identifiers.** Optima doesn't ask for school name, class group, student ID, or any teacher details. If the application ever did, it would intersect the NSW Department of Education's *Privacy Code of Practice*, which imposes stricter handling rules, a complication this application doesn't require.
- + **No persistent unique identifiers sent anywhere.** Even if a future addition were to add syncing to the cloud, the architecture would not use anything that could be correlated with a student record.

14. Glossary

Term	Meaning
14-day rotation	PMH's fortnightly timetable. Each class has a position in days 0-13.
Constraint	A fixed item in the 14-day rotation (a class/study period, or external commitment).
Cushion	The ratio of free study hours to the remaining workload hours in the next 14 days.
Critical override	The rule that any assignment due in ≤ 3 days jumps to priority 999.
FreeSlot	An interval of unoccupied time on a single date, used as a placement target by the scheduler.
Greedy placer	The scheduling strategy: rank tasks, fill the earliest free slot first, and repeat. Not optimal, but it remains deterministic and fast.
Per-user salt	A unique 16-byte string generated at signup and concatenated with the password before hashing.
Atomic write	Writing a file in a way that an interrupted write leaves either the old file or the new file on the disk and never a partial one.